
Rapport Final Projet DZ-CarPool

Module : Génie Logiciel
Niveau : 1CS Informatique

Plateforme web de covoiturage mettant en relation conducteurs et passagers DZ-CarPool

Réalisé par :

Bessah Rayane
Ouldchikh Larbi Yanis
Bekki Moustapha Aymen
Djoumer Yacine
Haddouche Athmane

Encadré par :

Mme. Behloul Fatiha

Remerciements

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet.

Nous remercions tout particulièrement **Mme. Behloul Fatiha**, notre encadrante, pour son accompagnement, ses conseils avisés et son soutien tout au long de ce projet. Sa disponibilité et son expertise ont été essentielles à la réussite de ce travail.

Nos remerciements vont également à l'ensemble du corps enseignant de l'**École Supérieure en Sciences et Technologies de l'Informatique et du Numérique de Béjaïa (ESTIN)** pour la formation de qualité qu'ils nous ont dispensée et qui nous a permis d'acquérir les compétences nécessaires pour mener à bien ce projet.

Nous remercions enfin nos familles et nos proches pour leur soutien indéfectible, leur patience et leurs encouragements constants durant toute la durée de ce projet.

L'équipe DZ-CarPool

Table des matières

Chapitre 1: Introduction	9
Chapitre 2: Présentation Générale du Projet	10
2.1 Description du Projet	12
2.2 Public Cible	12
2.3 Objectifs du Projet	12
2.4 Périmètre du MVP	12
Chapitre 3: Analyse des Besoins	12
3.1 Acteurs du Système	14
3.2 Besoins Fonctionnels	14
3.3 Besoins Non Fonctionnels	14
Chapitre 4: Conception du Système	14
4.1 Architecture Générale	16
4.2 Modélisation UML	16
Chapitre 5: Journal des Décisions d'Architecture	27
5.1 Choix du Framework Backend : Django	28
5.2 Choix de la Conteneurisation : Docker	28
5.3 Validation Manuelle des Réservations	29
5.4 Absence de Paiement en Ligne dans le MVP	29
5.5 Choix d'une Architecture REST	29
Chapitre 6: Implémentation	29
6.1 Technologies Utilisées	31
6.2 Fonctionnalités Implémentées	31
6.3 Structure Générale du Projet	31
6.3.1 Organisation Générale	31
6.3.2 Structure du Backend	32
6.3.3 Structure du Frontend	32
6.3.4 Conteneurisation	38
6.3.5 Intégration Continue	38
6.3.6 Documentation	39

Chapitre 7: Intégration de Données Externes	39
7.1 Prix du carburant	41
7.2 Intégration de l'API Google Maps	41
7.2.1 Principe de Fonctionnement	41
7.2.2 Sécurité et Avantages	41
Chapitre 8: Sécurité du Système	42
8.1 Chiffrement des Mots de Passe	44
8.2 Authentification par JWT	44
8.3 Connexion via Google OAuth2	45
8.4 Contrôle des Accès	45
8.5 Résumé des Mesures de Sécurité	45
Chapitre 9: Tests et Validation	46
9.1 Objectifs des Tests	48
9.2 Tests du Frontend	48
9.3 Organisation de la Suite de Tests BACKEND	49
9.4 Types de Tests Réalisés	50
9.4.1 Tests d'Authentification	50
9.4.2 Tests de Vérification	50
9.4.3 Tests de Profil	50
9.4.4 Tests de Documents	50
9.4.5 Tests de Sécurité	51
9.5 Couverture de Code	51
9.6 Intégration Continue	52
9.7 Tests d'Intégration et End-to-End (E2E)	52
9.7.1 Outil Utilisé	53
9.7.2 Scénarios Testés	53
9.7.3 Résultats des Tests	53
9.8 Synthèse des Résultats	54
9.9 Conclusion	54
Chapitre 10: Déploiement du Projet	54
10.1 Objectifs du Déploiement	56
10.2 Architecture de Déploiement	56
10.3 Déploiement du Frontend	56
10.4 Déploiement du Backend	57
10.5 Gestion du Temps Réel avec Redis Upstash	57
10.6 Bilan du Déploiement	57

Chapitre 11: Gestion de Projet	57
11.1 Méthodologie Adoptée	59
11.2 Organisation de l'Équipe	59
11.3 Sprint 1 : Initiation et Conception	59
11.4 Sprint 2 : Développement et Structuration	60
11.5 Sprint 3 : Intégration et Validation	60
11.6 Sprint 4 : Déploiement et Finalisation	61
11.7 Bilan Global	62
Chapitre 12: Conclusion	62
Chapitre 13: Annexes	64

Table des figures

4.1	Diagramme de cas d'utilisation qui illustre les interactions principales entre les acteurs (conducteur, passager, administrateur) et le système DZ-CarPool.	17
4.2	Diagramme de classes qui présente la structure statique du système, les classes principales, leurs attributs et les relations entre elles.	18
4.3	Diagramme de séquence qui décrit le déroulement des échanges entre les composants lors de la recherche d'un trajet par un utilisateur.	19
4.4	Diagramme de séquence qui montre le processus de communication entre un conducteur et un passager via le système.	20
4.5	Diagramme de séquence qui représente les étapes de l'authentification d'un utilisateur sur la plateforme.	21
4.6	Diagramme de séquence qui illustre le mécanisme d'envoi et de réception des notifications dans le système.	22
4.7	Diagramme de séquence qui montre les différentes étapes de création d'un compte utilisateur.	23
4.8	Diagramme de séquence qui décrit le processus de vérification des informations d'un utilisateur.	24
4.9	SCHEMA ER qui présente la structure de la base de données et les relations entre les entités principales.	25
4.10	Diagramme de composants illustrant l'organisation modulaire de l'application DZ-CarPool.	26
6.1	Page d'accueil de la plateforme DZ-CarPool présentant les options principales accessibles à l'utilisateur.	34
6.2	Page de connexion permettant aux utilisateurs de s'authentifier sur la plateforme DZ-CarPool.	35
6.3	Page d'inscription pour créer un compte utilisateur, avec saisie des informations personnelles et paramètres de sécurité.	35

6.4	Page de profil utilisateur affichant les informations personnelles, les trajets réservés et les paramètres du compte.	35
6.5	Interface utilisateur montrant le bouton pour publier une offre de trajet ou réserver une place disponible.	36
6.6	Page permettant à un utilisateur de publier un trajet sur la plateforme DZ-CarPool, avec les champs nécessaires pour le départ, l'arrivée, la date et le nombre de places disponibles.	36
6.7	Interface de réservation d'un trajet, où le passager peut sélectionner un trajet disponible, réserver une place et envoyer une demande au conducteur.	36
6.8	Vue listant les réservations effectuées par les passagers, avec détails sur les trajets et l'état de chaque réservation.	37
6.9	Page de recherche de trajets avec options de filtrage selon la date, la destination, le type de trajet et le nombre de places disponibles.	37
6.10	Section de notifications affichant les alertes et messages récents pour l'utilisateur connecté.	37
6.11	Dashboard de l'administrateur présentant un aperçu global des activités et statistiques de la plateforme DZ-CarPool.	38
6.12	Pipeline GitHub Actions - Vue d'ensemble des workflows automatisés	38
6.13	GitHub Actions - Détail de l'exécution des tests et vérifications	39
9.1	Rapport de couverture de code des services frontend	49
9.2	Résumé de la couverture de code frontend en ligne de commande	49
9.3	Résultats des tests End-to-End (Playwright) pour la création d'un trajet	54
A.1	Capture d'écran du backlog Jira - chronologie du sprint	72
A.1		72
A.2	Capture d'écran d'un backlog Jira - Détail du sprint 1	72
A.2		72
A.3	Capture d'écran d'un backlog Jira - Détail du sprint 2	72
A.3		72
A.4	Capture d'écran d'un backlog Jira - Détail du sprint 3	73
A.4		73
A.5	Capture d'écran d'un backlog Jira - Détail d'un sprint 4	73
A.5		73

Liste des Acronymes

MVP Minimum Viable Product

UML Unified Modeling Language

REST Representational State Transfer

API Application Programming Interface

JWT JSON Web Token

CI/CD Continuous Integration / Continuous Deployment

ORM Object-Relational Mapping

OAuth Open Authorization

UI User Interface

BDD Base de Données

Chapitre 1: Introduction

Le covoiturage est aujourd'hui une solution de mobilité durable permettant de réduire les coûts de déplacement, la congestion routière ainsi que l'impact environnemental. Dans un contexte où les déplacements inter-wilayas sont fréquents et coûteux, la mise en relation efficace entre conducteurs et passagers devient essentielle.

Le projet **DZ-CarPool** s'inscrit dans cette perspective en proposant une plateforme web dédiée au covoiturage en Algérie.

Objectifs de l'Introduction : Cette section détaille le contexte, les enjeux et les objectifs principaux de la plateforme. L'utilisateur doit comprendre la finalité du projet, son utilité sociale et économique ainsi que les contraintes auxquelles il répond.

Contexte : Le développement du projet DZ-CarPool s'inscrit dans une tendance mondiale de promotion de la mobilité durable et du partage de transport. En Algérie, les déplacements inter-wilayas sont fréquents et souvent coûteux, rendant nécessaire une solution numérique pour faciliter la mise en relation entre conducteurs et passagers.

Enjeux :

- Réduction de la congestion et de l'empreinte carbone.
- Réduction des coûts de transport pour les utilisateurs.
- Création d'une interface simple et fiable pour le covoiturage.
- Respect des contraintes légales et sécuritaires pour les transactions et les données utilisateurs.

Présentation du rapport : Ce rapport présente l'ensemble des étapes du projet, depuis l'analyse des besoins jusqu'aux choix techniques et organisationnels, en incluant la conception, l'implémentation et les tests.

Chapitre 2: Présentation Générale du Projet

2.1 Description du Projet

DZ-CarPool est une plateforme web de covoiturage qui met en relation des conducteurs disposant de places libres dans leurs véhicules avec des passagers souhaitant effectuer le même trajet. La plateforme permet la publication de trajets, la recherche ciblée, la réservation de places et la communication entre utilisateurs.

2.2 Public Cible

Le projet s'adresse principalement :

- Aux conducteurs effectuant des trajets réguliers ou occasionnels
- Aux passagers cherchant une solution de transport économique

2.3 Objectifs du Projet

- Faciliter les déplacements inter-wilayas
- Réduire les coûts de transport
- Encourager le partage et la solidarité
- Proposer une solution numérique simple et fiable

2.4 Périmètre du MVP

Le projet se concentre sur un **Minimum Viable Product (MVP)** intégrant uniquement les fonctionnalités essentielles : gestion des utilisateurs, publication d'offres de covoiturage, recherche et consultation des offres, réservation et contact.

Chapitre 3: Analyse des Besoins

Introduction

Ce chapitre détaille les besoins du système DZ-CarPool. Il identifie les différents acteurs, les besoins fonctionnels et non fonctionnels, ainsi que les contraintes à prendre en compte pour garantir le bon fonctionnement et la qualité du service proposé.

3.1 Acteurs du Système

- **Conducteur** : publie des trajets et valide les réservations
- **Passager** : recherche et réserve des trajets
- **Administrateur** : supervise la plateforme (optionnel)

3.2 Besoins Fonctionnels

- Inscription et authentification des utilisateurs
- Gestion du profil utilisateur
- Publication, modification et suppression de trajets
- Recherche de trajets selon différents critères
- Réservation de places avec validation par le conducteur
- Messagerie entre conducteur et passager

3.3 Besoins Non Fonctionnels

- Sécurité des données et des accès
- Performance et temps de réponse acceptable
- Disponibilité de la plateforme
- Maintenabilité et évolutivité du système

Chapitre 4: Conception du Système

Introduction

Le chapitre de conception décrit l'architecture générale de la plateforme et la modélisation UML des composants, des classes et des séquences d'interaction. Il permet de comprendre la structure interne du système et la manière dont les différents modules communiquent entre eux.

4.1 Architecture Générale

L'application adopte une architecture **client-serveur** basée sur des services **REST**. Le frontend communique avec le backend via des API sécurisées, permettant une séparation claire des responsabilités.

4.2 Modélisation UML

Plusieurs diagrammes UML ont été réalisés afin de modéliser le système.

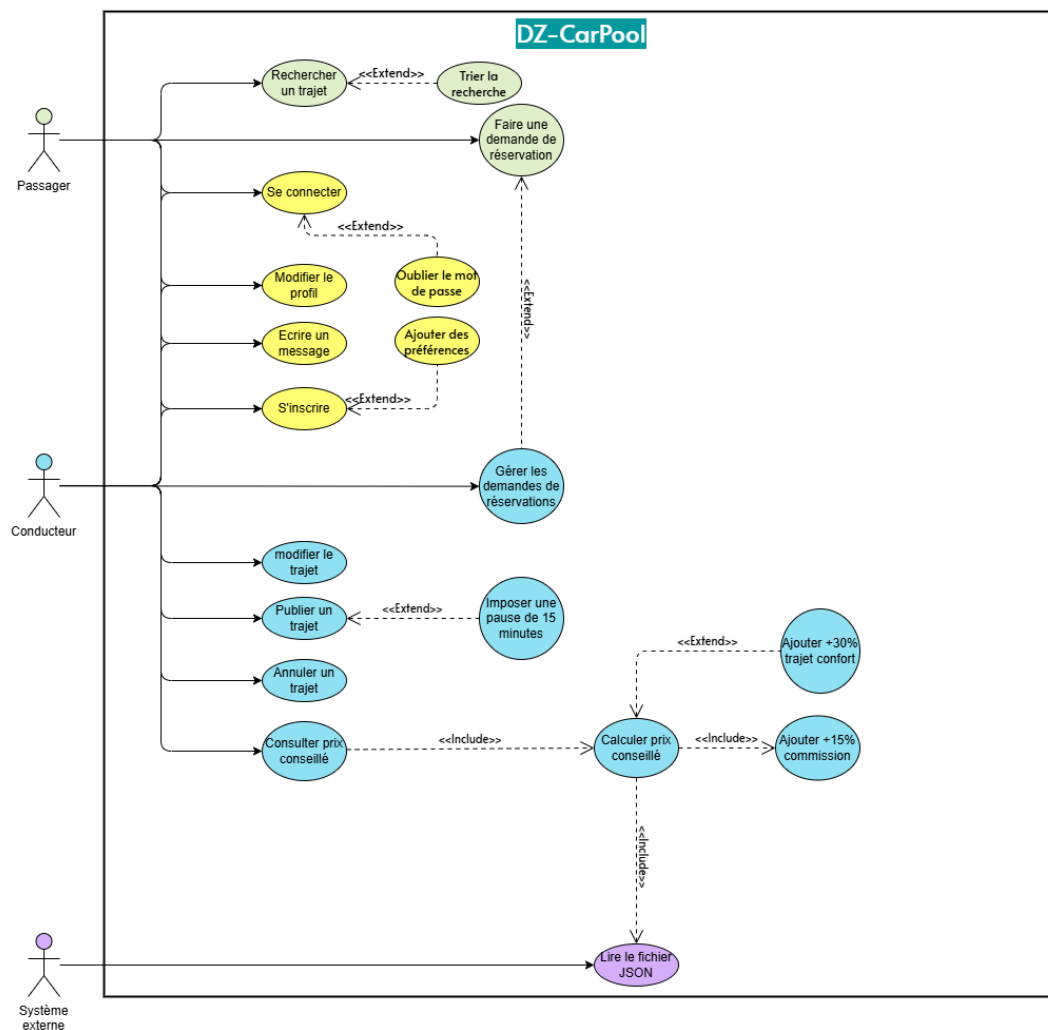


Fig. 4.1 : Diagramme de cas d'utilisation qui illustre les interactions principales entre les acteurs (conducteur, passager, administrateur) et le système DZ-CarPool.

Diagramme de classe - DZ-CARPOOL

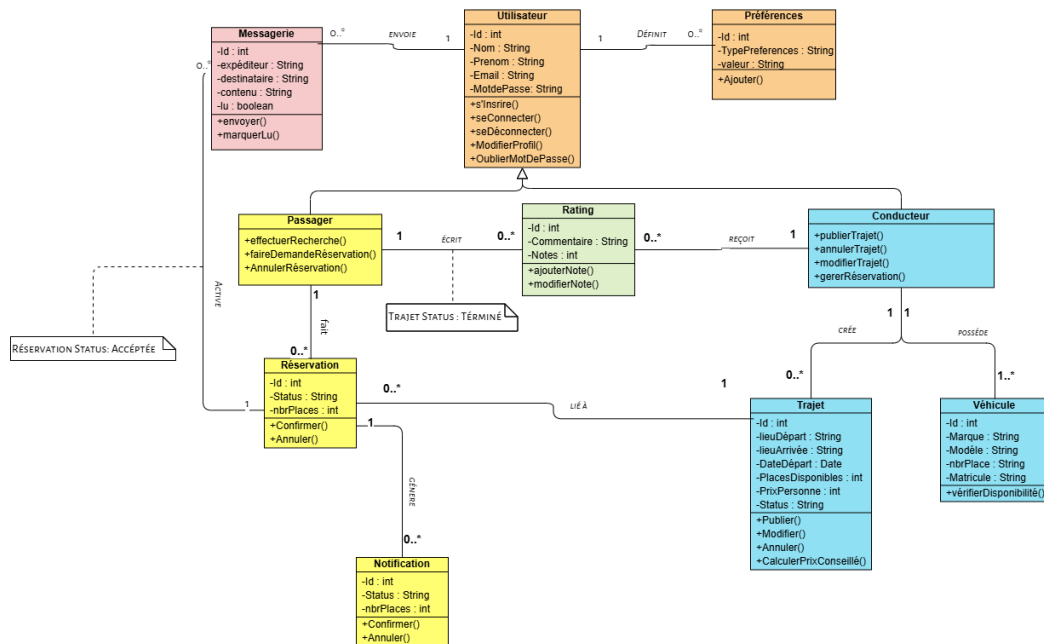


Fig. 4.2 : Diagramme de classes qui présente la structure statique du système, les classes principales, leurs attributs et les relations entre elles.

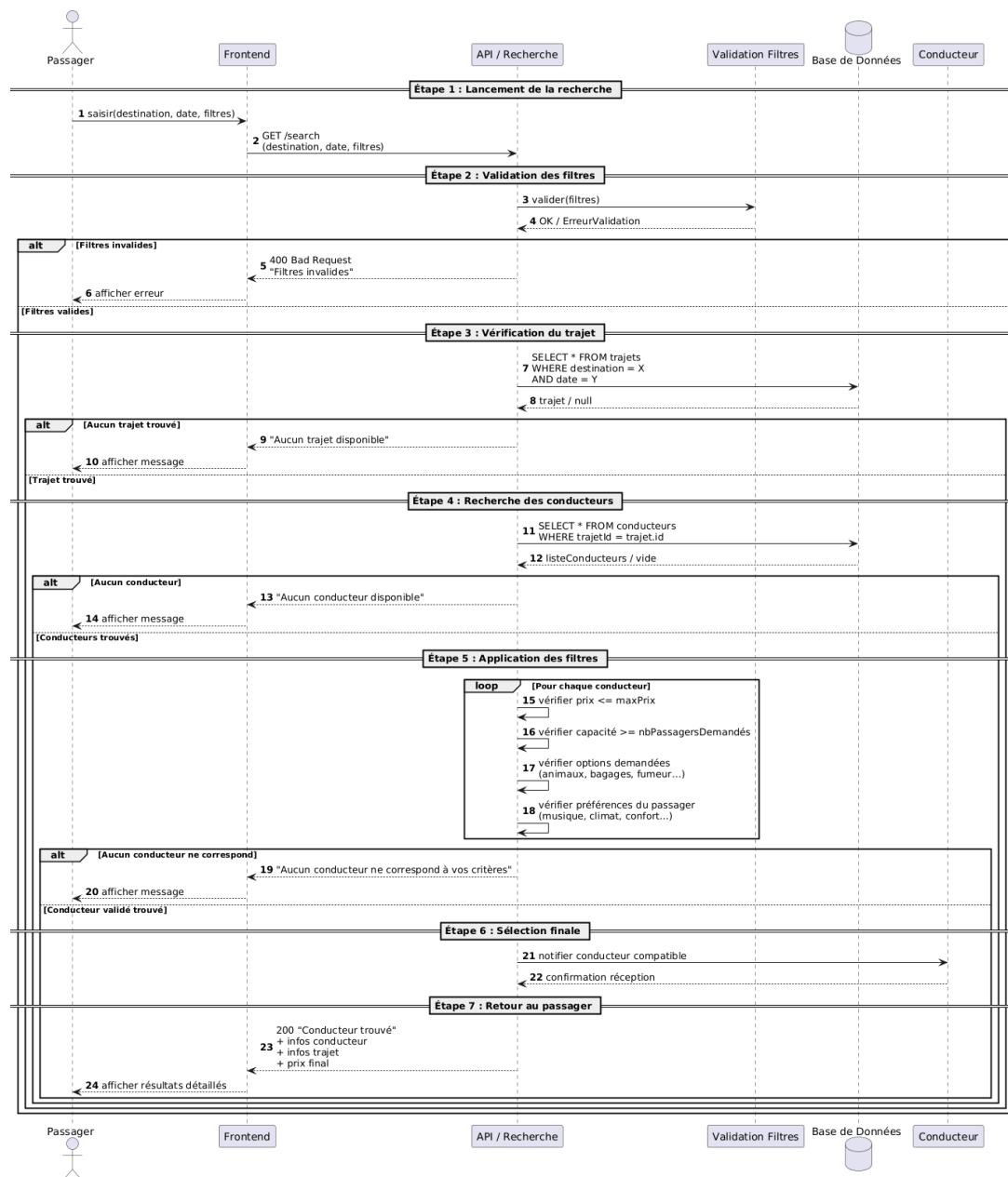


Fig. 4.3 : Diagramme de séquence qui décrit le déroulement des échanges entre les composants lors de la recherche d'un trajet par un utilisateur.

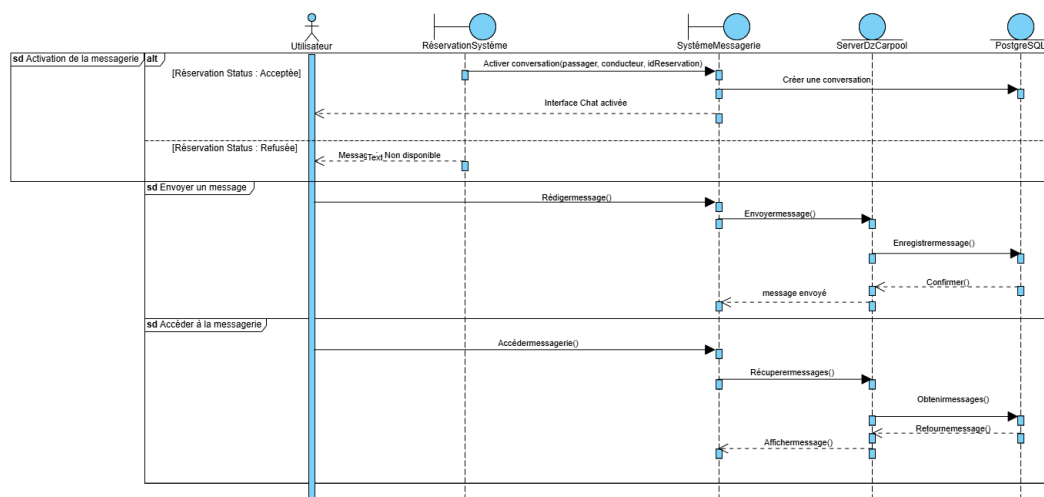


Fig. 4.4 : Diagramme de séquence qui montre le processus de communication entre un conducteur et un passager via le système.

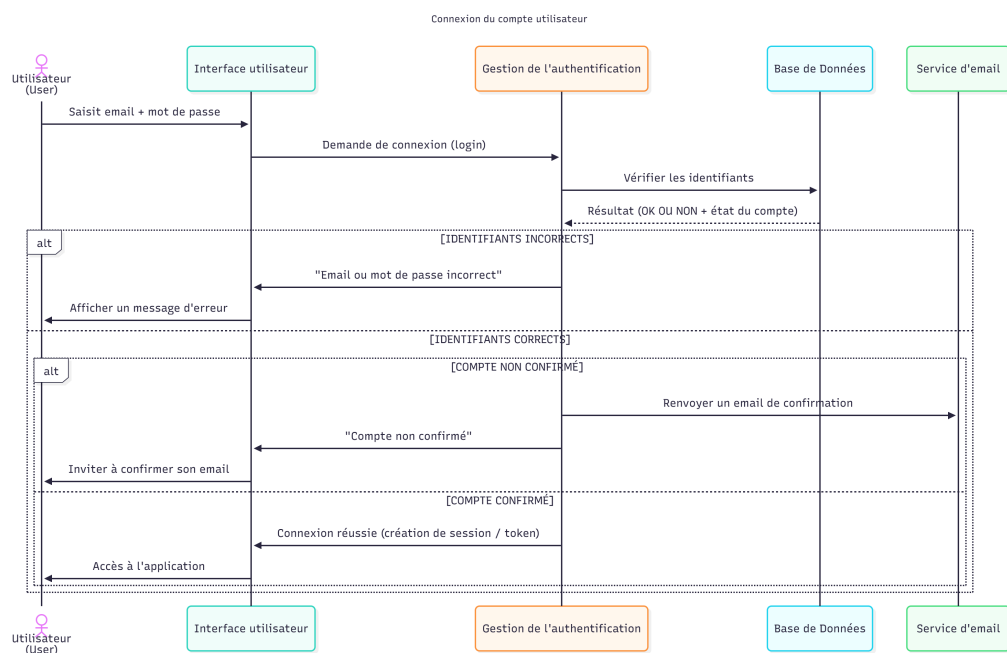


Fig. 4.5 : Diagramme de séquence qui représente les étapes de l'authentification d'un utilisateur sur la plateforme.

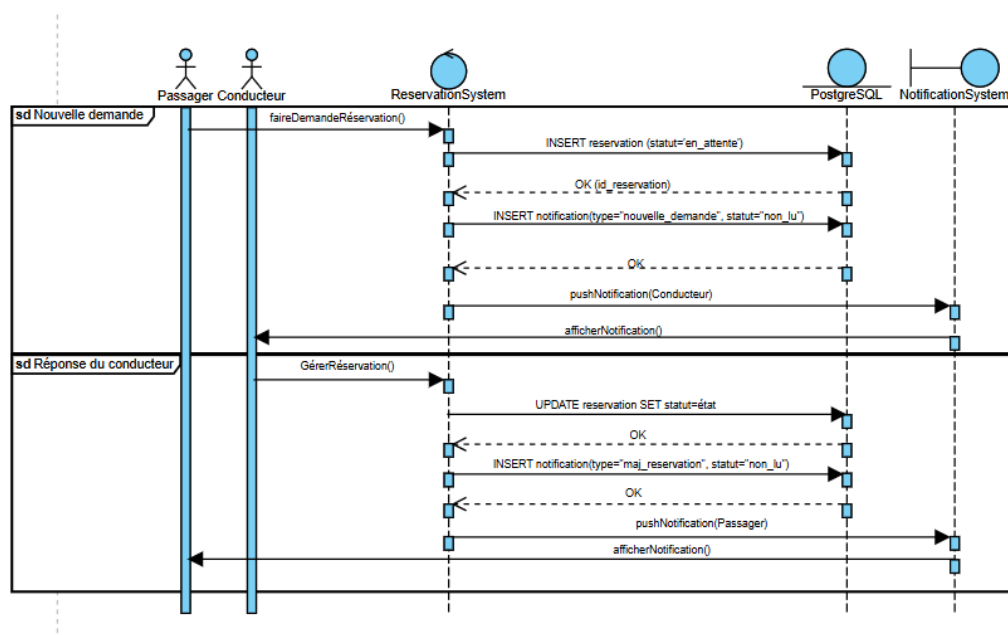


Fig. 4.6 : Diagramme de séquence qui illustre le mécanisme d'envoi et de réception des notifications dans le système.

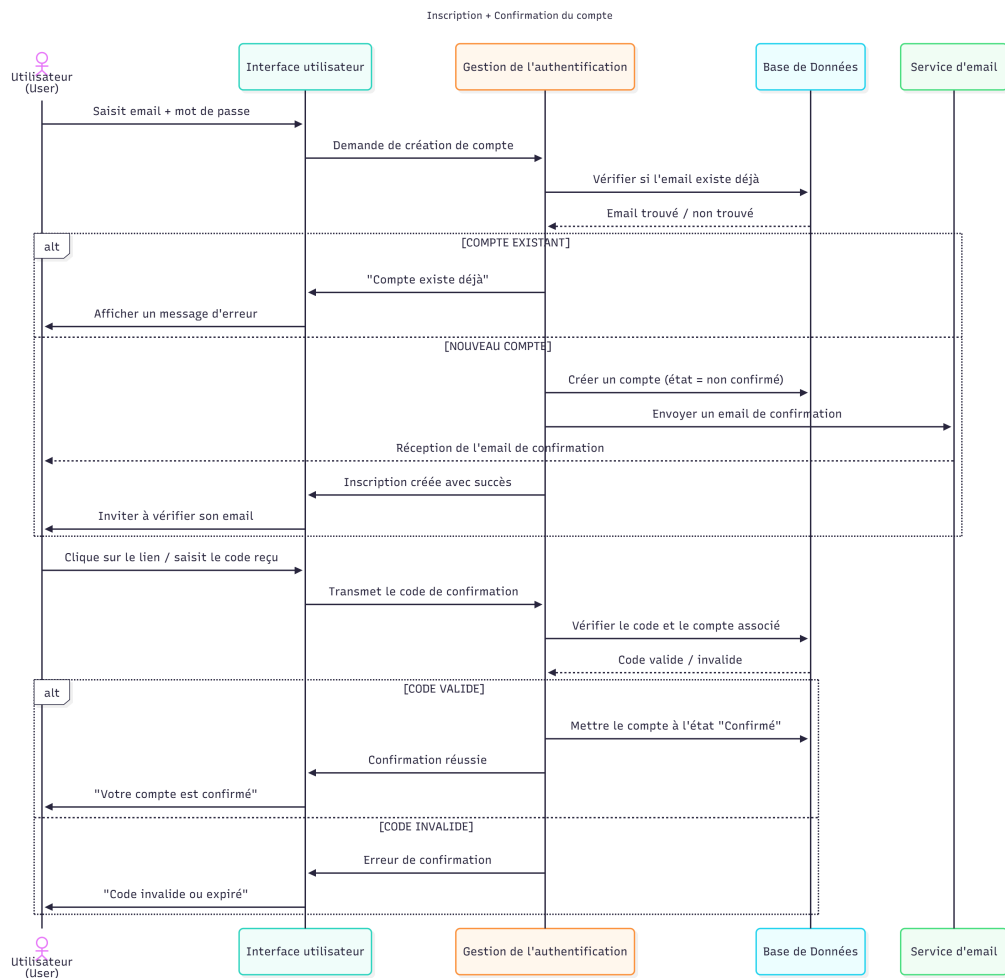


Fig. 4.7 : Diagramme de séquence qui montre les différentes étapes de création d'un compte utilisateur.

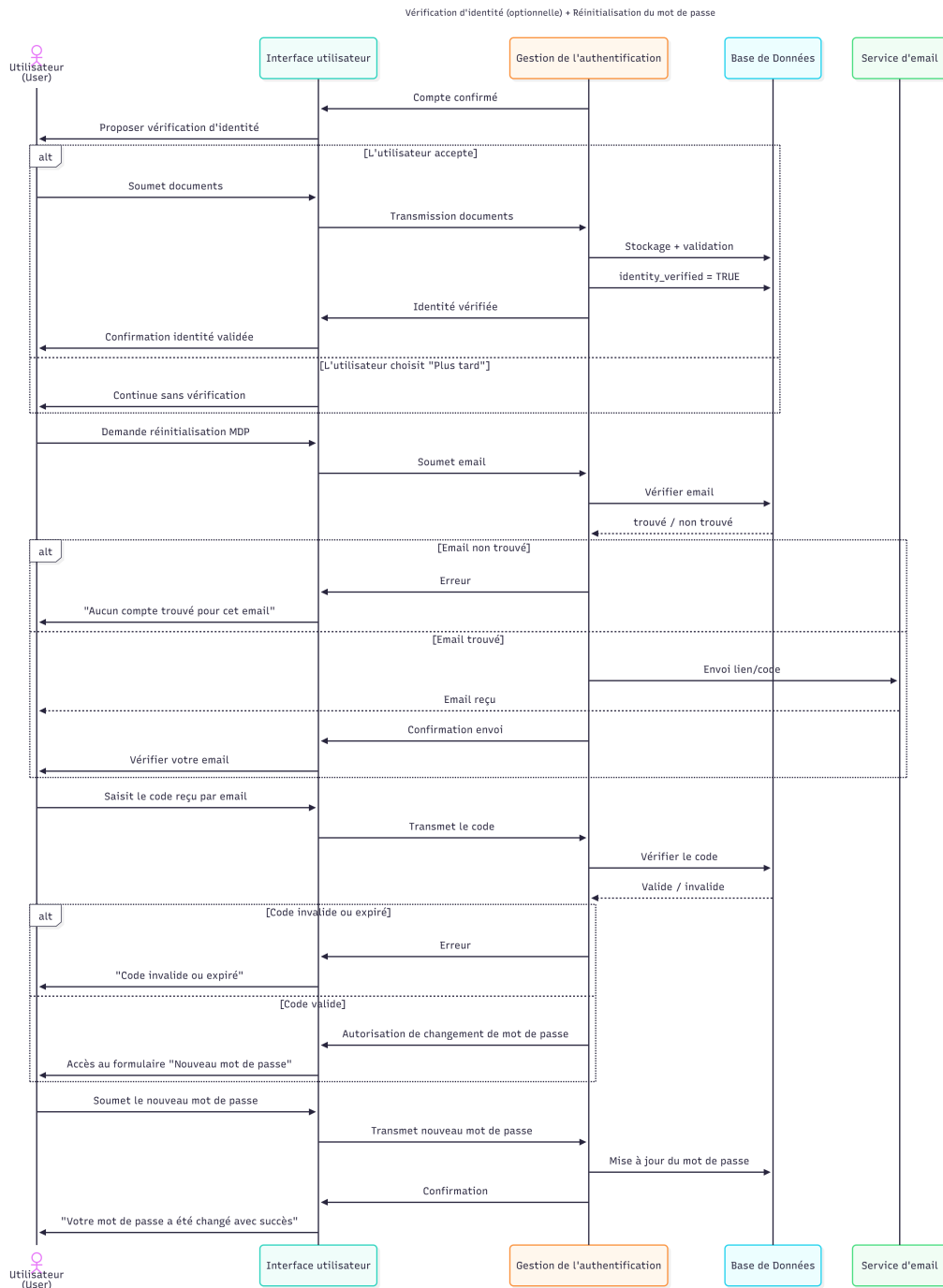


Fig. 4.8 : Diagramme de séquence qui décrit le processus de vérification des informations d'un utilisateur.

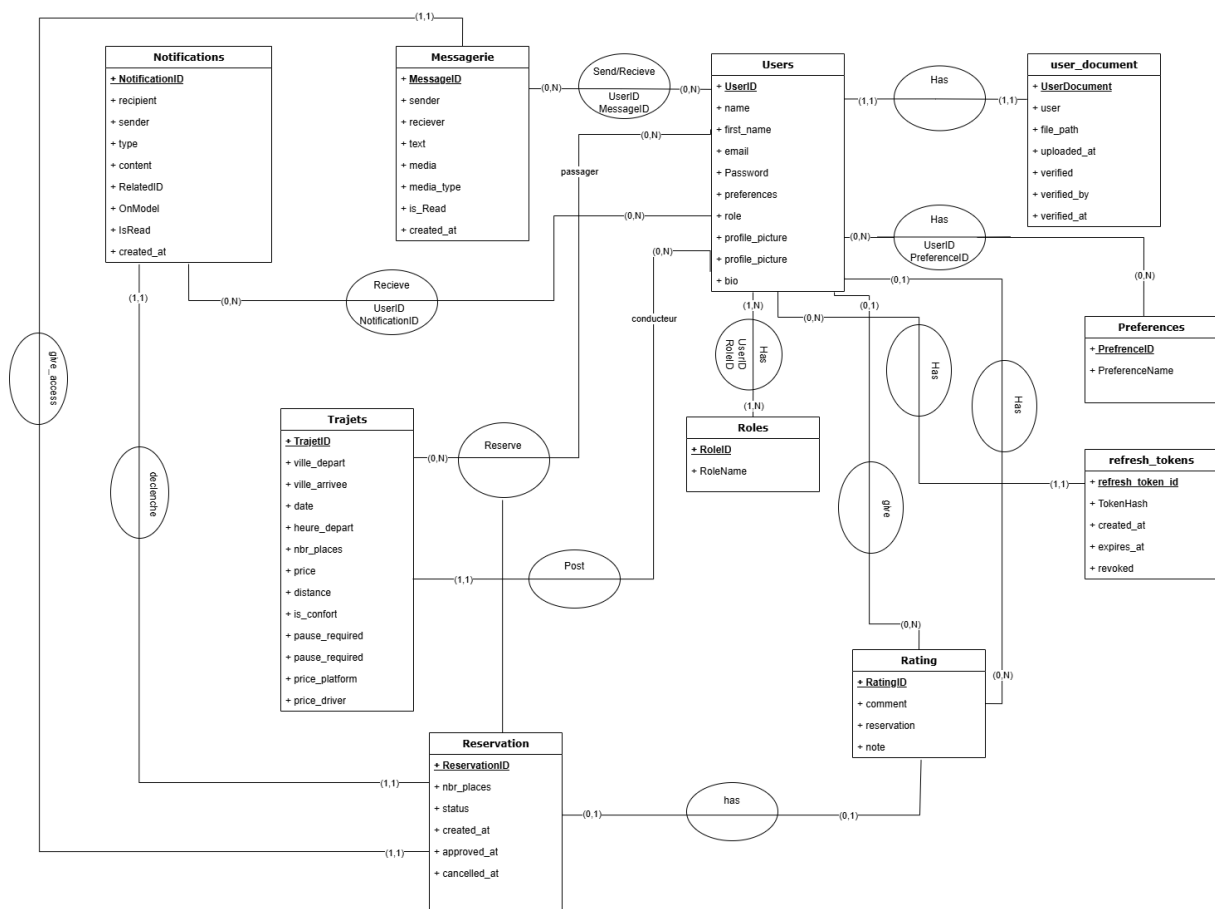


Fig. 4.9 : SCHEMA ER qui présente la structure de la base de données et les relations entre les entités principales.

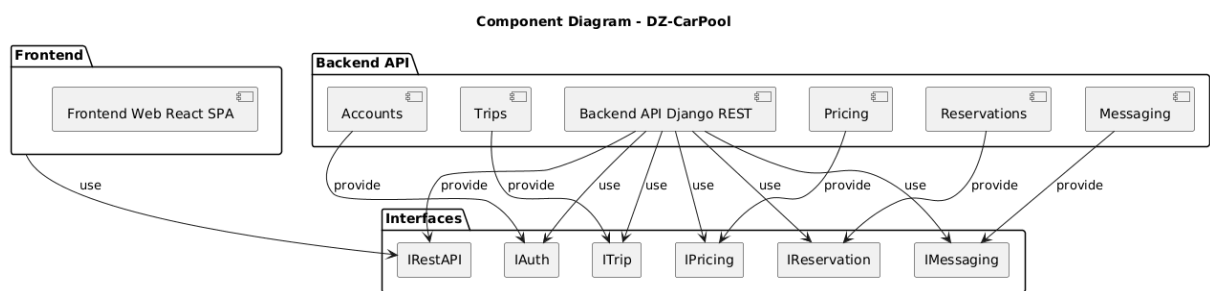


Fig. 4.10 : Diagramme de composants illustrant l'organisation modulaire de l'application DZ-CarPool.

Chapitre 5: Journal des Décisions d'Architecture

Introduction

Ce chapitre présente le journal d'architecture du projet DZ-CarPool. Il retrace les principales décisions architecturales prises tout au long du développement, les choix technologiques, les compromis effectués et les évolutions de l'architecture. Ce journal permet de comprendre la logique derrière la conception du système et d'assurer la traçabilité des décisions pour faciliter la maintenance et l'évolution future du projet.

5.1 Choix du Framework Backend : Django

Le choix de Django a été motivé par plusieurs raisons :

- **sécurité intégrée** (gestion utilisateurs, protection CSRF, hachage des mots de passe),
- **ORM puissant** facilitant la manipulation des données,
- **structure claire et modulaire** adaptée aux projets académiques,
- large communauté et documentation riche.

Ce choix permet d'obtenir une base robuste et fiable tout en réduisant les risques liés aux erreurs de sécurité.

5.2 Choix de la Conteneurisation : Docker

Docker a été utilisé afin de :

- garantir la reproductibilité des environnements,
- faciliter le déploiement,
- isoler les dépendances,
- standardiser la configuration entre les membres de l'équipe.

Ainsi, le projet peut être exécuté de manière identique en développement et en production.

5.3 Validation Manuelle des Réservations

Deux approches ont été envisagées :

- réservation automatique,
- réservation avec validation par le conducteur.

La seconde a été retenue afin de :

- éviter les réservations abusives,
- laisser au conducteur la maîtrise de son véhicule,
- limiter les conflits entre utilisateurs.

5.4 Absence de Paiement en Ligne dans le MVP

Le paiement en ligne n'a pas été retenu dans le périmètre du MVP pour les raisons suivantes :

- complexité légale et sécuritaire,
- gestion des litiges et remboursements,
- besoin de conformité bancaire,
- augmentation importante de la charge technique.

Cette fonctionnalité pourra être ajoutée dans une version ultérieure du produit.

5.5 Choix d'une Architecture REST

L'API suit les principes REST afin de :

- séparer frontend et backend,
- permettre une future application mobile,
- garantir une structure claire des échanges,
- faciliter les tests et l'intégration continue.

Chapitre 6: Implémentation

Introduction

Ce chapitre présente la mise en œuvre technique du projet DZ-CarPool. Il détaille les technologies utilisées, l'organisation du backend et du frontend, la conteneurisation avec Docker, l'intégration continue et la documentation. Il montre comment la conception a été traduite en un système opérationnel.

6.1 Technologies Utilisées

- Backend : Django
- Base de données : PostgreSQL
- Frontend : React
- Gestion de versions : Git / GitHub
- Conteneurisation : Docker

6.2 Fonctionnalités Implémentées

Toutes les fonctionnalités du MVP ont été implémentées.

6.3 Structure Générale du Projet

Le projet DZ-CarPool adopte une organisation modulaire facilitant la maintenance et l'évolutivité.

6.3.1 Organisation Générale

- Fichiers Docker et CI/CD
- Documentation technique
- Gestion de versions Git

6.3.2 Structure du Backend

Développé avec Django, structuré en applications, services et tests.

Backend/

app/

core/

management/

commands/

messaging/

migrations/

notifications/

migrations/

...

reservations/

migrations/

...

trips/

migrations/

...

users/

migrations/

...

settings.py

urls.py

wsgi.py

manage.py

requirements.txt

Dockerfile

6.3.3 Structure du Frontend

Basé sur React avec composants réutilisables.

Frontend/

app/

- about/
- admin/
- careers/
- contact/
- documents/
- find-ride/
- forgot-password/
- login/
- messages/
- notifications/
- offer-ride/
- preferences/
- profile/
- reservations/
 - [id]/
- reset-password/
- search-results/
- signup/
- trip/
 - [id]/
- trips/
- verify/

- components/
 - ui/
- contexts/
- hooks/
- lib/
- services/
- stores/
- styles/
- public/
 - images/
- __mocks__/_

```
__tests__/  
  app/  
  components/  
  helpers/  
  hooks/  
  integration/  
  services/  
  stores/  
coverage/  
  app/  
  components/  
  hooks/  
  services/  
  stores/  
  lcov-report/  
.swc/  
  plugins/  
    windows_x86_64_20.0.0/
```



Fig. 6.1 : Page d'accueil de la plateforme DZ-CarPool présentant les options principales accessibles à l'utilisateur.

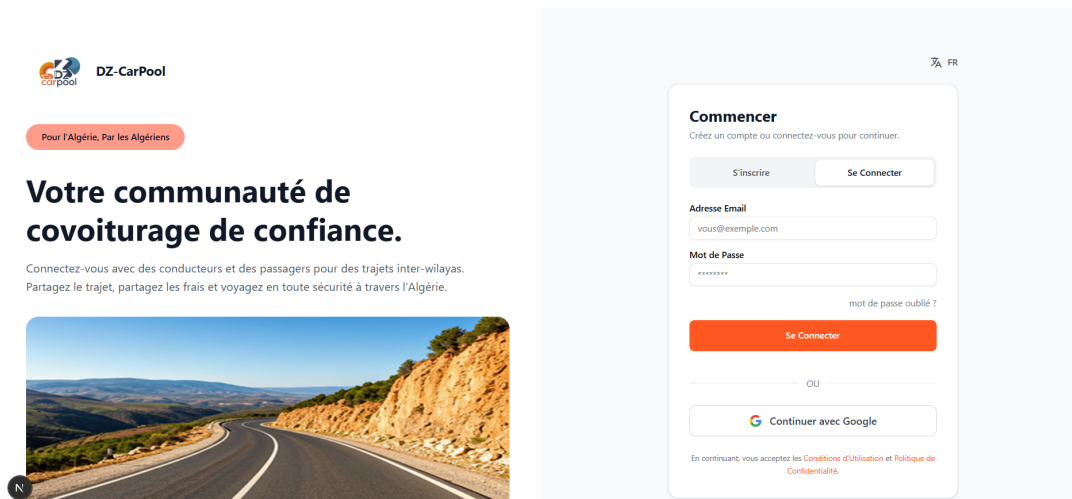


Fig. 6.2 : Page de connexion permettant aux utilisateurs de s'authentifier sur la plateforme DZ-CarPool.

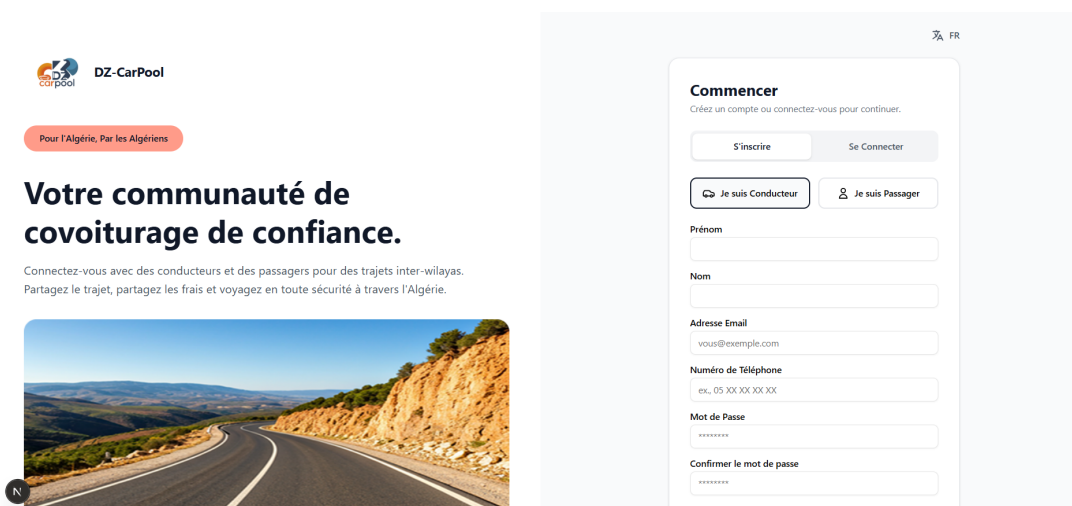


Fig. 6.3 : Page d'inscription pour créer un compte utilisateur, avec saisie des informations personnelles et paramètres de sécurité.

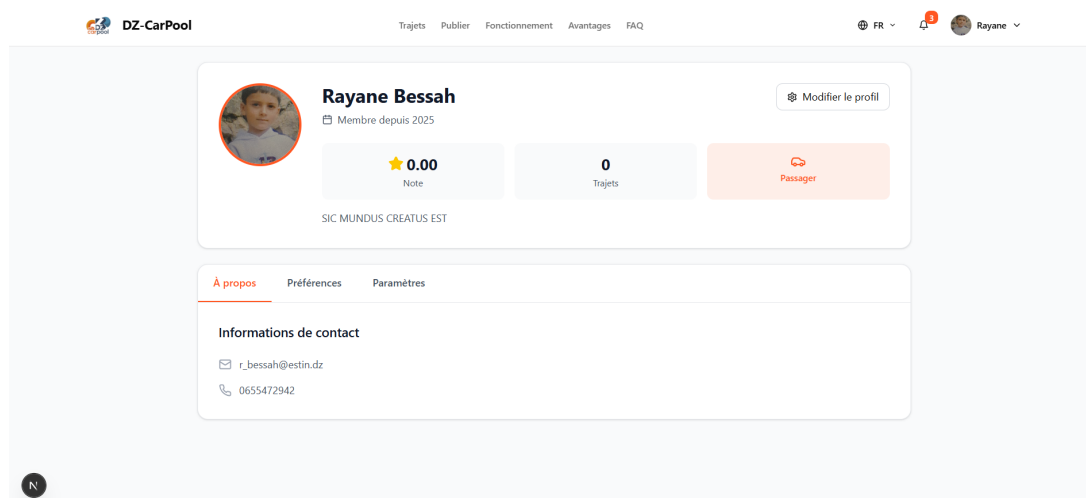


Fig. 6.4 : Page de profil utilisateur affichant les informations personnelles, les trajets réservés et les paramètres du compte.

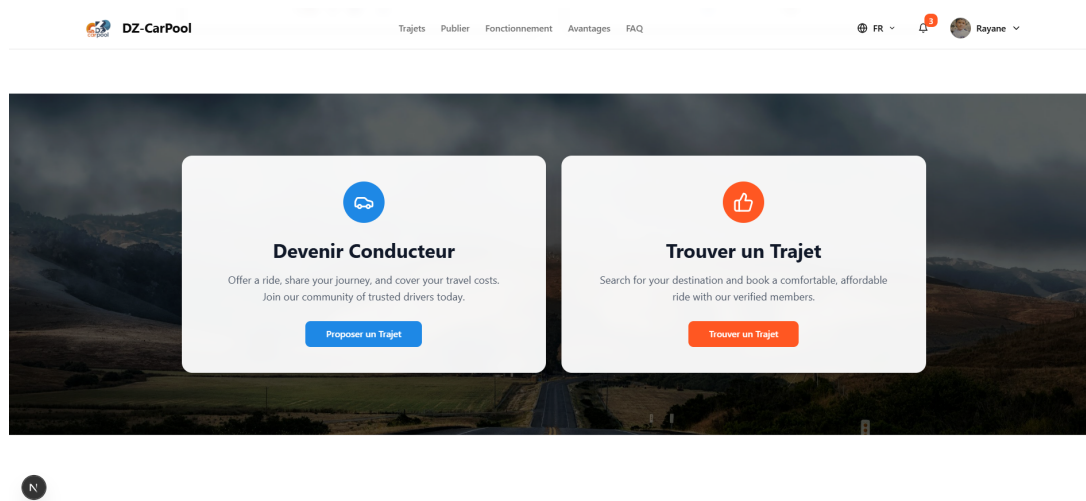


Fig. 6.5 : Interface utilisateur montrant le bouton pour publier une offre de trajet ou réserver une place disponible.

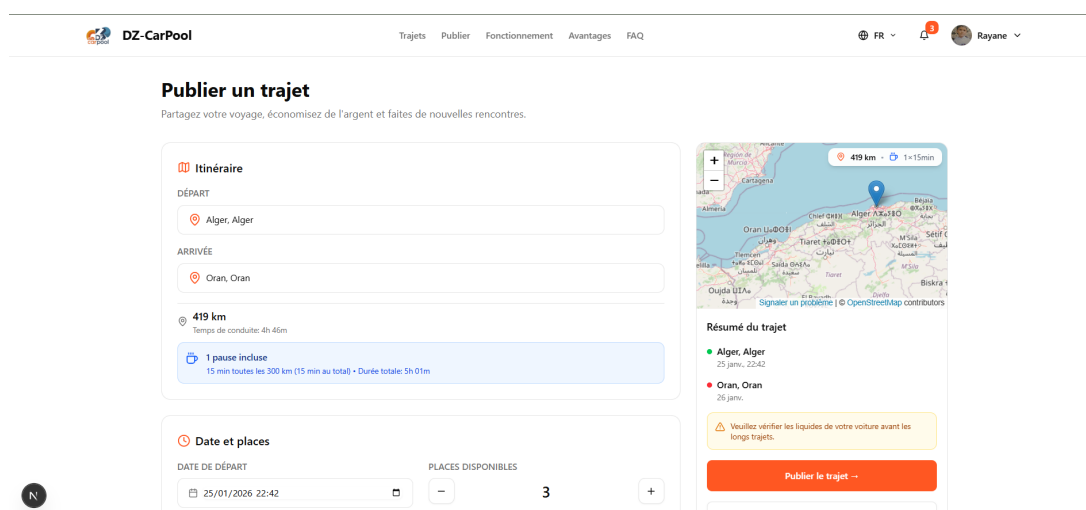


Fig. 6.6 : Page permettant à un utilisateur de publier un trajet sur la plateforme DZ-CarPool, avec les champs nécessaires pour le départ, l'arrivée, la date et le nombre de places disponibles.

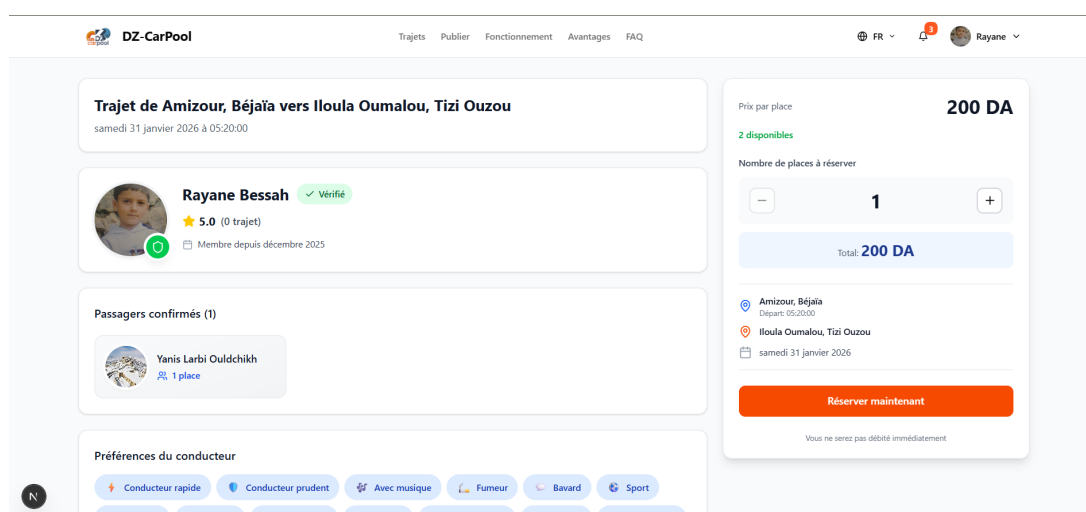


Fig. 6.7 : Interface de réservation d'un trajet, où le passager peut sélectionner un trajet disponible, réserver une place et envoyer une demande au conducteur.

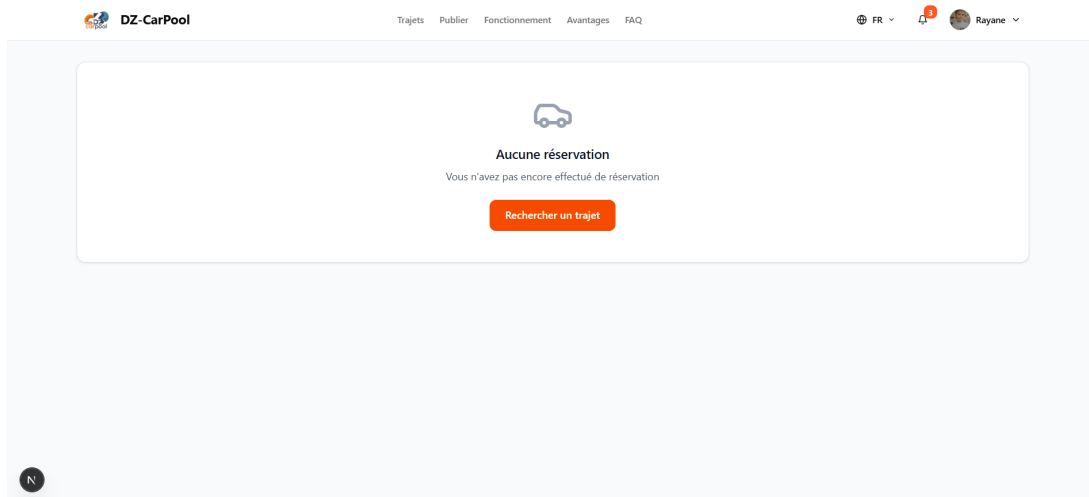


Fig. 6.8 : Vue listant les réservations effectuées par les passagers, avec détails sur les trajets et l'état de chaque réservation.

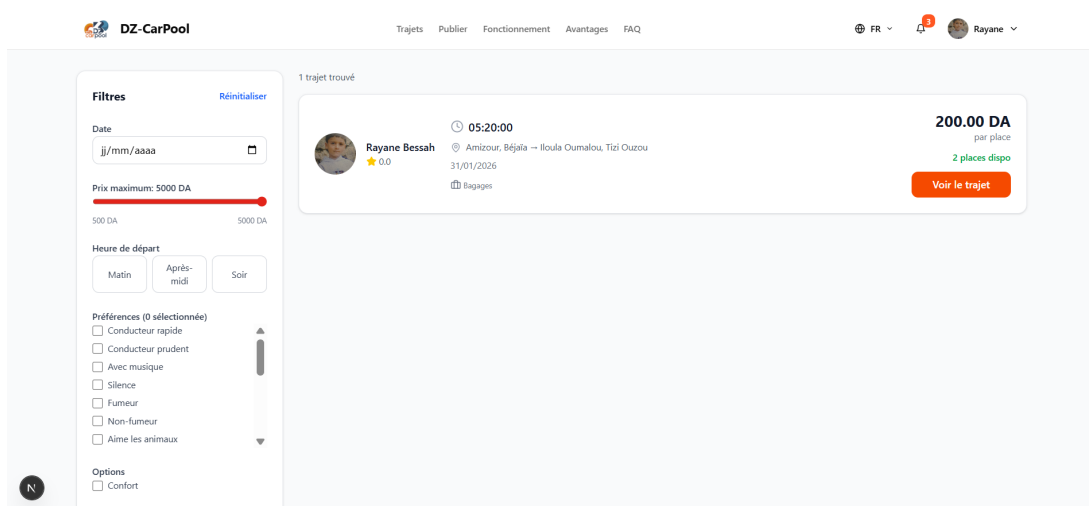


Fig. 6.9 : Page de recherche de trajets avec options de filtrage selon la date, la destination, le type de trajet et le nombre de places disponibles.



Fig. 6.10 : Section de notifications affichant les alertes et messages récents pour l'utilisateur connecté.

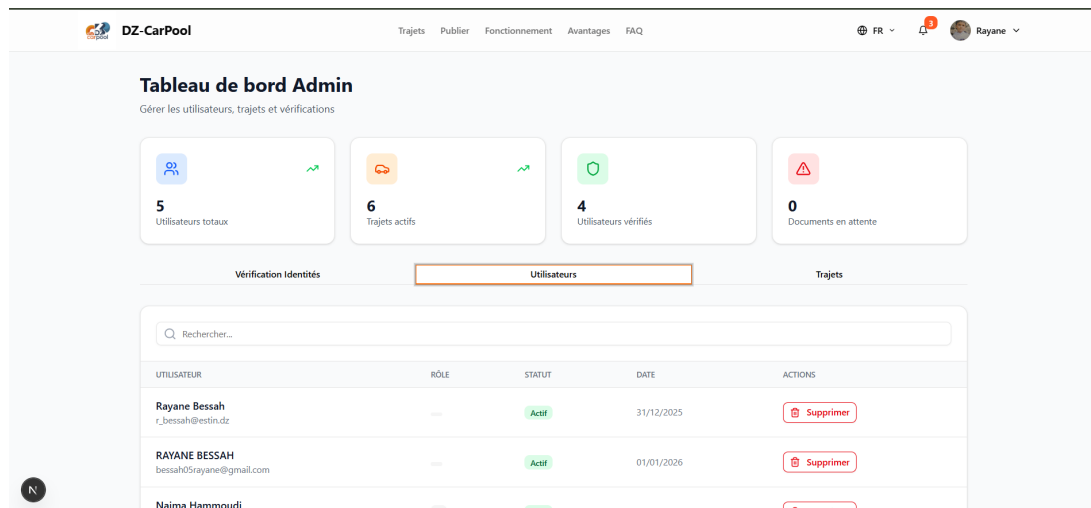


Fig. 6.11 : Dashboard de l'administrateur présentant un aperçu global des activités et statistiques de la plateforme DZ-CarPool.

6.3.4 Conteneurisation

Utilisation de Docker pour les environnements dev/prod.

6.3.5 Intégration Continue

Pipeline GitHub Actions pour automatiser les tests.

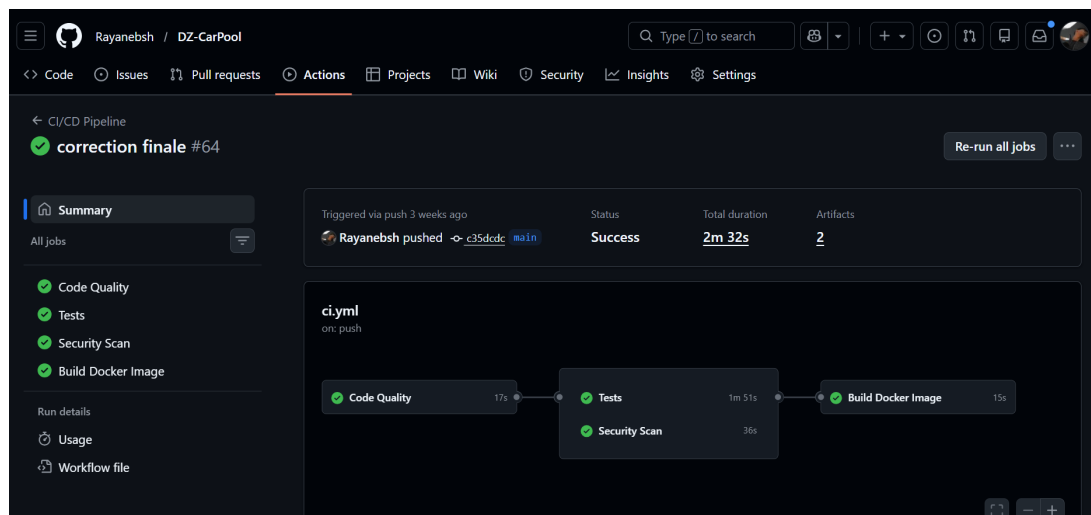


Fig. 6.12 : Pipeline GitHub Actions - Vue d'ensemble des workflows automatisés

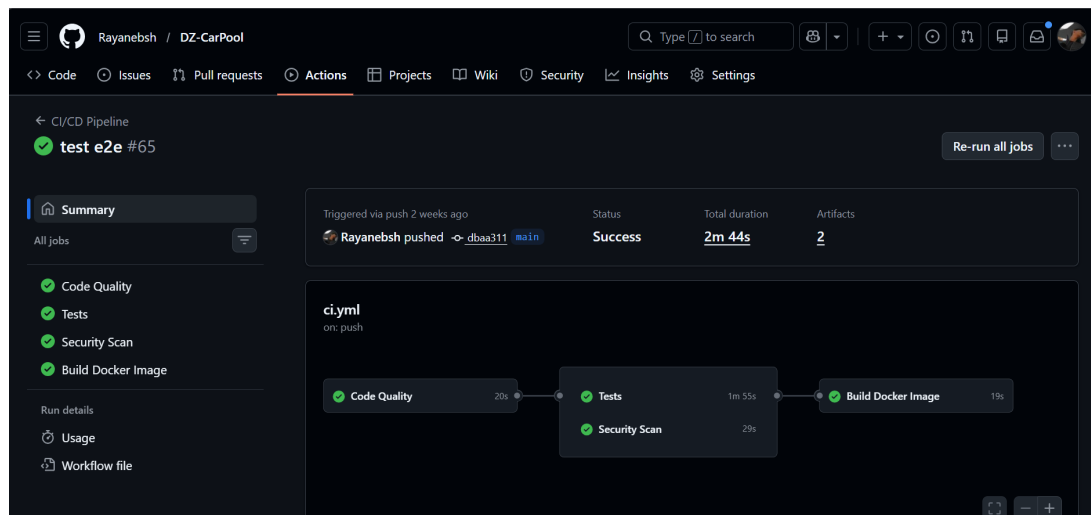


Fig. 6.13 : GitHub Actions - Détail de l'exécution des tests et vérifications

6.3.6 Documentation

Dossier docs/ pour l'architecture et le déploiement.

Chapitre 7: Intégration de Données Externes

Introduction

Dans ce chapitre, nous expliquons comment DZ-CarPool intègre des données externes pour améliorer son fonctionnement. Il présente l'utilisation de fichiers de prix du carburant et l'API Google Maps pour calculer automatiquement les distances, afficher les trajets et enrichir l'expérience utilisateur.

7.1 Prix du carburant

Lors de la création d'un trajet, le système lit le fichier `prix_carburants.json`. Le calcul du prix conseillé est basé sur :

- Une consommation moyenne de 8L / 100 km
- Le prix du carburant
- Le nombre de places disponibles

7.2 Intégration de l'API Google Maps

En complément des données locales relatives aux prix des carburants, la plateforme DZ-CarPool intègre l'API **Google Maps** afin d'enrichir la gestion des trajets et d'améliorer l'expérience utilisateur.

Cette API est utilisée pour l'affichage interactif des cartes, la visualisation des itinéraires, ainsi que le calcul automatique de la distance entre le point de départ et la destination lors de la création d'un trajet.

7.2.1 Principe de Fonctionnement

Lorsqu'un conducteur saisit les lieux de départ et d'arrivée, le système interagit avec Google Maps pour déterminer l'itinéraire optimal et la distance totale. Cette distance est ensuite exploitée pour le calcul du prix suggéré du trajet et pour l'affichage de l'aperçu cartographique.

7.2.2 Sécurité et Avantages

L'accès à l'API est sécurisé par une clé stockée dans les variables d'environnement, avec des restrictions de domaine et de quotas afin de prévenir les abus.

L'intégration de Google Maps permet :

- une meilleure visualisation des trajets,
- une estimation précise des distances,
- une automatisation fiable du calcul des prix,
- une réduction des erreurs de saisie.

Cette intégration constitue un élément clé de la fiabilité fonctionnelle de la plateforme DZ-CarPool.

Chapitre 8: Sécurité du Système

Introduction

Ce chapitre décrit les mesures de sécurité mises en place pour protéger les données et les accès des utilisateurs. Il couvre le chiffrement des mots de passe, l'authentification par JWT, l'intégration OAuth2, le contrôle des permissions et les bonnes pratiques appliquées pour garantir un système sûr.

La sécurité constitue un élément fondamental de la plateforme DZ-CarPool. Plusieurs mécanismes techniques ont été intégrés afin de protéger les données des utilisateurs et garantir un accès sécurisé au système.

8.1 Chiffrement des Mots de Passe

Les mots de passe des utilisateurs ne sont jamais stockés en clair dans la base de données. Ils sont hachés à l'aide d'algorithmes robustes tels que **PBKDF2**, **bcrypt** ou **Argon2**.

Chaque mot de passe est accompagné d'une *salt* unique, ce qui empêche l'utilisation de tables arc-en-ciel et rend les attaques par force brute beaucoup plus difficiles.

8.2 Authentification par JWT

L'authentification repose sur l'utilisation de **JSON Web Tokens (JWT)**.

Principe de fonctionnement

1. L'utilisateur saisit ses identifiants.
2. Si l'authentification réussit, le serveur génère un token signé.
3. Ce token contient l'identifiant de l'utilisateur et une date d'expiration.
4. Le token est transmis au client et stocké côté navigateur.
5. À chaque requête protégée, le token est envoyé dans l'en-tête HTTP.

Le serveur vérifie la validité du token avant d'accorder l'accès aux ressources protégées.

Ce mécanisme permet une gestion d'authentification **stateless**, réduisant la charge serveur tout en maintenant un haut niveau de sécurité.

8.3 Connexion via Google OAuth2

La plateforme supporte également l'authentification via Google en utilisant le protocole **OAuth2**.

Étapes principales

1. L'utilisateur choisit «Se connecter avec Google».
2. Il est redirigé vers la page officielle Google.
3. Google vérifie son identité.
4. Un jeton sécurisé est renvoyé à DZ-CarPool.
5. Le serveur vérifie ce jeton et crée (ou associe) le compte utilisateur.

Ainsi, le mot de passe de l'utilisateur n'est jamais partagé avec notre système.

8.4 Contrôle des Accès

Un contrôle strict des permissions est appliqué :

- un utilisateur ne peut consulter que ses propres données,
- les actions critiques nécessitent une authentification valide,
- certains rôles (ex. administrateur) disposent de privilèges supplémentaires.

Toutes les entrées sont validées afin de réduire les risques liés aux injections.

8.5 Résumé des Mesures de Sécurité

- Hachage sécurisé des mots de passe
- Authentification via JWT
- Support OAuth2 via Google
- Validation et filtrage des données entrantes

- Isolation des données utilisateur

Ces mécanismes garantissent un niveau de sécurité conforme aux bonnes pratiques actuelles du développement web.

Chapitre 9: Tests et Validation

Introduction

Ce chapitre présente la stratégie de tests appliquée sur DZ-CarPool. Il détaille les tests unitaires, les tests d'intégration, les tests End-to-End, la couverture de code et l'intégration continue, afin de vérifier que le système fonctionne correctement et répond aux besoins définis.

9.1 Objectifs des Tests

Les tests ont été mis en place afin de vérifier :

- la conformité aux besoins fonctionnels,
- la robustesse du backend,
- la sécurité et le contrôle des accès,
- la stabilité lors des mises à jour.

L'environnement de test repose sur **pytest** et **pytest-django**.

9.2 Tests du Frontend

Des tests automatisés ont été mis en place pour le frontend afin de vérifier le bon fonctionnement des services métiers et garantir la stabilité de l'application.

Les tests ont été réalisés à l'aide d'outils de test JavaScript, permettant de mesurer la couverture de code selon plusieurs critères : instructions, branches, fonctions et lignes de code.

La figure suivante présente un rapport global de couverture de code pour les services du frontend.

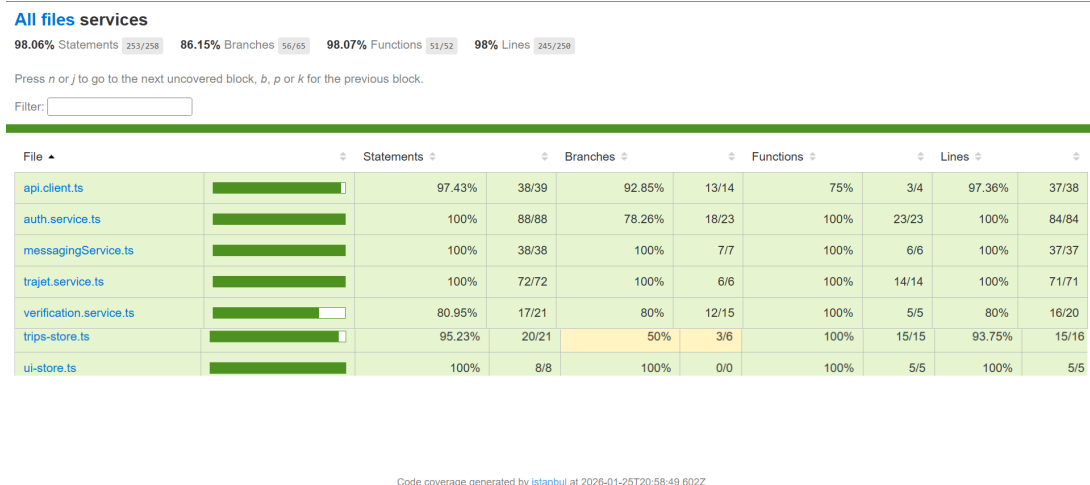


Fig. 9.1 : Rapport de couverture de code des services frontend

File	% Stmts	% Branch	% Funcs	% Lines
services	98.06	86.15	98.07	98
api.client.ts	97.43	92.85	75	97.36
auth.service.ts	100	78.26	100	100
messagingService.ts	100	100	100	100
trajet.service.ts	100	100	100	100
verification.service.ts	80.95	80	100	80

Fig. 9.2 : Résumé de la couverture de code frontend en ligne de commande

Les résultats obtenus montrent une couverture de code élevée, avec plus de 98% des instructions et des lignes couvertes. Ces indicateurs témoignent de la fiabilité des services frontend et de la bonne prise en compte des différents scénarios d'exécution.

9.3 Organisation de la Suite de Tests BACKEND

La structure suivante a été adoptée :

```
app/users/tests/
  __init__.py
  conftest.py
  test_auth.py
  test_verification.py
  test_profile.py
  test_documents.py
  test_security.py
```

Chaque fichier couvre un module fonctionnel précis.

9.4 Types de Tests Réalisés

9.4.1 Tests d'Authentification

Ils valident notamment :

- l'inscription,
- la connexion,
- la validation des champs,
- la gestion des erreurs,
- l'authentification Google.

9.4.2 Tests de Vérification

- envoi et validation des codes email,
- vérification du numéro de téléphone,
- gestion des codes expirés.

9.4.3 Tests de Profil

- récupération des informations utilisateur,
- mise à jour du profil,
- changement de mot de passe.

9.4.4 Tests de Documents

- upload sécurisé,
- validation des formats,
- respect des permissions d'accès.

9.4.5 Tests de Sécurité

Ils assurent que :

- les mots de passe sont correctement hachés,
- seuls les utilisateurs authentifiés accèdent aux ressources protégées,
- les tokens JWT invalides sont rejetés,
- un utilisateur ne peut pas consulter les données d'un autre.

9.5 Couverture de Code

File ↗	Statements				Branches			Total
	coverage	statements	missing	excluded	coverage	branches	partial	coverage
app \ notifications \ models.py	100.00%	76	0	12	87.50%	8	1	98.81%
app \ notifications \ serializers.py	100.00%	15	0	0	100.00%	2	0	100.00%
app \ notifications \ signals.py	91.89%	37	3	0	66.67%	18	4	83.64%
app \ notifications \ urls.py	100.00%	7	0	0	100.00%	0	0	100.00%
app \ notifications \ utils.py	90.57%	53	5	0	100.00%	0	0	90.57%
app \ notifications \ views.py	53.66%	41	19	0	0.00%	2	0	51.16%
app \ reservations \ models.py	100.00%	80	0	4	100.00%	12	0	100.00%
app \ reservations \ permissions.py	92.86%	14	1	0	83.33%	6	1	90.00%
app \ reservations \ serializers.py	83.33%	96	16	0	81.25%	16	3	83.04%
app \ reservations \ signals.py	89.26%	121	13	0	77.27%	22	5	87.41%
app \ reservations \ urls.py	100.00%	6	0	0	100.00%	0	0	100.00%
app \ reservations \ views.py	87.76%	196	24	0	89.13%	46	5	88.02%
app \ trajets \ models.py	92.71%	96	7	8	83.33%	6	1	92.16%
app \ trajets \ permissions.py	92.31%	39	3	0	83.33%	24	4	88.89%
app \ trajets \ serializers.py	86.84%	114	15	0	50.00%	24	6	80.43%
app \ trajets \ urls.py	100.00%	8	0	0	100.00%	0	0	100.00%
app \ trajets \ views.py	83.73%	295	48	0	73.53%	68	16	81.82%
app \ users \ models.py	95.76%	165	7	14	80.00%	20	4	94.05%
app \ users \ permissions.py	100.00%	0	0	0	100.00%	0	0	100.00%
app \ users \ serializers.py	97.44%	117	3	0	83.33%	24	4	95.04%
app \ users \ services.py	89.66%	29	3	0	100.00%	4	0	90.91%
app \ users \ urls.py	100.00%	12	0	0	100.00%	0	0	100.00%
app \ users \ views.py	81.76%	296	54	0	65.15%	66	3	78.73%
Total	88.45%	1913	221	38	75.82%	368	57	86.41%

coverage.py v7.13.0, created at 2026-01-02 00:41 +0100

Une couverture cible de **90 %** a été visée.

La couverture réelle a été mesurée automatiquement via :

```
pytest --cov=app --cov-report=html
```

Le rapport généré (htmlcov/index.html) présente les résultats suivants :

- **88.45 % de couverture des statements,**
- **75.82 % de couverture des branches,**

- soit une **couverture globale de 86.41 %**.

La majorité des modules dépasse **90 %**, notamment :

- notifications,
- reservations,
- `users.models`,
- `users.serializers`.

Les valeurs plus faibles concernent principalement certaines vues API, ce qui est cohérent avec la complexité des cas métiers et les différents scénarios utilisateurs.

9.6 Intégration Continue

Les tests sont exécutés automatiquement via **GitHub Actions**.

Le pipeline inclut :

1. analyse statique du code,
2. exécution des tests unitaires et API,
3. vérification de sécurité,
4. génération du rapport de couverture,
5. construction de l'image Docker,
6. déploiement si succès.

9.7 Tests d'Intégration et End-to-End (E2E)

En complément des tests unitaires et des tests API, des tests d'intégration de type **End-to-End (E2E)** ont été mis en place afin de valider le bon fonctionnement global de l'application du point de vue utilisateur.

Ces tests simulent le comportement réel d'un utilisateur en interagissant directement avec l'interface web (frontend React) et en vérifiant l'intégration complète avec le backend Django et la base de données.

9.7.1 Outil Utilisé

Les tests E2E ont été implémentés à l'aide de l'outil **Playwright**, exécuté sur le navigateur **Chromium**. Playwright permet de tester automatiquement les scénarios utilisateurs critiques dans un environnement proche de la production.

9.7.2 Scénarios Testés

Les scénarios suivants ont été automatisés et validés :

- Création complète d'un trajet avec succès
- Validation des champs obligatoires
- Calcul automatique de la distance
- Calcul du prix suggéré basé sur le carburant
- Option de trajet *Confort* (+30%)
- Sélection des préférences de trajet
- Aperçu de l'itinéraire sur la carte
- Ajout de détails supplémentaires
- Restriction des dates aux dates futures uniquement

Chaque scénario correspond à un cas de test indépendant, garantissant la robustesse fonctionnelle de la fonctionnalité **Création de trajet**.

9.7.3 Résultats des Tests

L'ensemble des tests E2E a été exécuté avec succès, comme illustré dans la figure ci-dessous. Tous les scénarios ont été validés sans échec, confirmant la bonne intégration entre le frontend, le backend et les services externes (cartographie, calcul de distance).

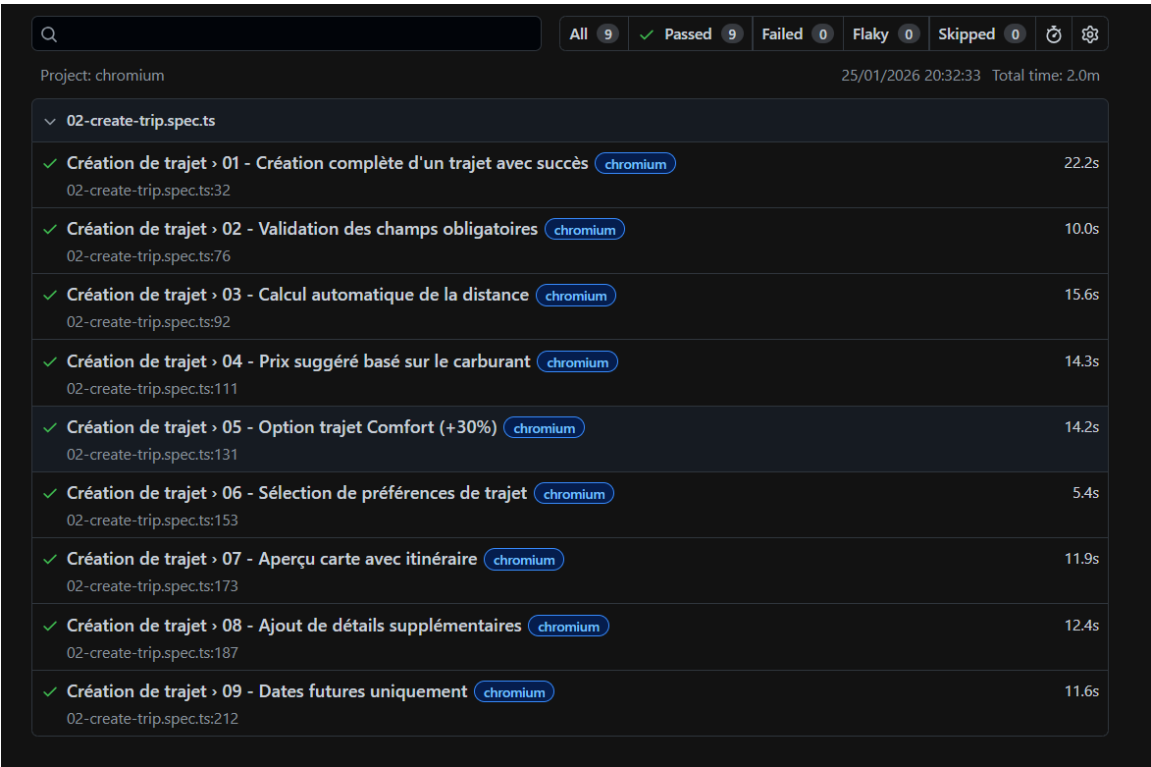


Fig. 9.3 : Résultats des tests End-to-End (Playwright) pour la création d’un trajet

Ces résultats démontrent la stabilité de l’application et la conformité du parcours utilisateur aux exigences fonctionnelles définies dans le cahier des charges.

9.8 Synthèse des Résultats

Catégorie	Objectif	Résultat
Unitaires	Logique métier	Validé
API	Endpoints REST	Validé
Sécurité	Authentification / Accès	Validé
Fonctionnels	Parcours utilisateur	Validé
Couverture	≥ 85%	86.41%

9.9 Conclusion

Les tests effectués démontrent la stabilité, la fiabilité et la conformité du système aux exigences définies. Avec une couverture de code supérieure à **86 %**, le projet présente un niveau de qualité satisfaisant et une base solide pour les futures évolutions.

Chapitre 10: Déploiement du Projet

Introduction

Le chapitre de déploiement explique comment la plateforme DZ-CarPool a été rendue accessible en ligne. Il décrit l'architecture de déploiement, le positionnement du frontend et du backend, ainsi que la gestion du temps réel avec Redis, assurant une disponibilité stable et fiable de la plateforme.

10.1 Objectifs du Déploiement

Le déploiement du projet DZ-CarPool vise à rendre la plateforme accessible en ligne dans un environnement stable et proche de la production, tout en assurant la séparation des responsabilités entre les composants du système.

10.2 Architecture de Déploiement

La plateforme adopte une architecture distribuée, reposant sur des services d'hébergement spécialisés :

- le **frontend React** est déployé sur **Vercel**,
- le **backend Django** est déployé sur **Render**,
- la base de données est hébergée séparément,
- la gestion des communications temps réel repose sur **Redis Upstash**.

Cette séparation améliore la scalabilité et la maintenabilité du système.

10.3 Déploiement du Frontend

Le frontend est déployé sur **Vercel**, une plateforme optimisée pour les applications web modernes. Chaque mise à jour du dépôt déclenche automatiquement un nouveau déploiement, garantissant une mise à jour rapide de l'interface utilisateur.

10.4 Déploiement du Backend

Le backend Django est hébergé sur **Render**. Il expose une API REST sécurisée utilisée par le frontend. Les variables sensibles (clés API, secrets JWT, accès base de données) sont gérées via les variables d'environnement de la plateforme.

10.5 Gestion du Temps Réel avec Redis Upstash

Les fonctionnalités temps réel, notamment la messagerie et les notifications, s'appuient sur **Redis Upstash**. Ce service est utilisé comme *broker* pour la communication via WebSocket, permettant une synchronisation efficace entre les utilisateurs connectés.

10.6 Bilan du Déploiement

Ce déploiement permet de valider le fonctionnement complet de DZ-CarPool dans des conditions réelles d'utilisation. Il constitue une base fiable pour une future montée en charge et l'ajout de nouvelles fonctionnalités.

Chapitre 11: Gestion de Projet

Introduction

Dans ce chapitre, nous présentons l'organisation de l'équipe et la méthodologie Scrum utilisée pour mener le projet. Il décrit les différents sprints, leurs objectifs, le rôle de chaque membre et les livrables réalisés, permettant de comprendre le déroulement global du projet et la coordination des activités.

11.1 Méthodologie Adoptée

Le projet DZ-CarPool a été mené selon la méthodologie **Scrum**. Cette approche Agile repose sur des cycles de développement courts appelés **sprints**, permettant une adaptation continue et une amélioration progressive du produit.

11.2 Organisation de l'Équipe

L'équipe est composée de cinq membres. Un **chef de sprint** est désigné à chaque sprint afin d'assurer la coordination, le suivi de l'avancement et la répartition équitable des tâches.

11.3 Sprint 1 : Initiation et Conception

Chef de sprint : Bessah Rayane

Ce sprint s'est déroulé du **27 novembre 2025 au 30 novembre 2025**. Il avait pour objectif la mise en place des bases du projet.

Objectifs

- Validation du sujet DZ-CarPool
- Définition de l'architecture du système
- Conception des diagrammes UML
- Mise en place de l'environnement de développement

Travaux Réalisés

- Diagramme de cas d'utilisation

- Diagramme de classes
- Diagrammes de séquence
- Conception de la base de données
- Premières maquettes UI

11.4 Sprint 2 : Développement et Structuration

Chef de sprint : Ould Chikh Larbi Yanis

Ce sprint s'est déroulé du **1 décembre 2025 au 14 décembre 2025**. Il était dédié au développement du système.

Objectifs

- Développement frontend et backend
- Finalisation du diagramme de composants
- Enrichissement du design

Travaux Réalisés

- Développement de plusieurs pages frontend
- Avancement significatif du backend
- Ajout de nouvelles pages de design
- Validation du diagramme de composants

11.5 Sprint 3 : Intégration et Validation

Chef de sprint : Haddouche Athmane

Ce sprint s'est déroulé du **15 décembre 2025 au 04 janvier 2026**. Il avait pour objectif la finalisation technique et documentaire du projet, ainsi que l'intégration complète entre les différents modules.

Objectifs

- Finaliser le design et le frontend
- Intégrer entièrement le backend et le frontend
- Documenter les décisions d'architecture
- Rédiger la documentation technique et académique
- Mettre en place et exécuter les tests

Travaux Réalisés

- Rédaction du document ARCHITECTURE_DECISIONS.md justifiant les choix techniques.
- Implémentation du frontend (profil, administration, notifications, messagerie) et finalisation du design global.
- Intégration complète entre le backend Django et le frontend React, avec correction des derniers détails.
- Rédaction du rapport final de Génie Logiciel.
- Rédaction du rapport de base de données.
- Mise en place et exécution des tests unitaires et d'intégration, ainsi que de l'intégration continue.

11.6 Sprint 4 : Déploiement et Finalisation

Chef de sprint : Djoumer Yacine

Sprint du **5 janvier au 25 janvier 2026**, consacré au déploiement et aux derniers ajustements du projet.

Objectifs

- Déployer la plateforme
- Poursuivre les tests et corriger les anomalies

- Finaliser l'interface et les rapports
- Réaliser une vidéo de présentation du projet

Travaux Réalisés

- Déploiement effectif de DZ-CarPool
- Tests utilisateurs et corrections de bugs
- Retouches sur l'interface
- Réalisation et publication d'une vidéo de déploiement sur YouTube.
- Mise à jour du rapport final et du rapport BDD.
- Deux réunions en présentiel (12/01 pour les tâches, 24/01 pour l'avancement).

Contraintes

- Le déroulement du projet a été perturbé par des contraintes externes (grève du transport) ayant limité la coordination de l'équipe. Suite à ces perturbations et au report de la date de présentation, la décision a été prise d'ajouter un quatrième sprint afin d'assurer la finalisation complète du projet.
- Les examens de fin de semestre ont ralenti temporairement le travail. Malgré la fatigue, l'équipe a su reprendre un bon rythme vers la fin du sprint.

11.7 Bilan Global

La méthodologie Scrum a permis une bonne organisation et une progression claire du projet. Les quatre sprints ont structuré le travail de la conception jusqu'au déploiement.

Malgré la charge académique, l'équipe est restée impliquée et solidaire. Le Sprint 4 a été décisif avec le déploiement, la finalisation des livrables et la présentation du projet.

DZ-CarPool est ainsi un MVP fonctionnel, testé et documenté, illustrant concrètement l'application du génie logiciel et du travail en équipe.

Chapitre 12: Conclusion

Ce projet nous a permis de mettre en pratique les concepts fondamentaux du génie logiciel sur un cas réel. DZ-CarPool constitue aujourd'hui une base fonctionnelle solide, offrant la possibilité de gérer efficacement les trajets, les réservations et la communication entre utilisateurs.

Cependant, cette version initiale peut être améliorée et enrichie par l'ajout de fonctionnalités supplémentaires, telles que les paiements en ligne, les notifications en temps réel ou une application mobile dédiée, afin de rendre l'expérience utilisateur plus pratique et complète.

Ainsi, DZ-CarPool représente un prototype fonctionnel et évolutif, qui illustre l'application concrète des méthodes de développement logiciel tout en ouvrant la voie à des extensions futures.

Chapitre 13: Annexes

DZ-CarPool - Script SQL

Script SQL - DZ-CarPool (Page 1/6)

```
-- =====
-- SCRIPT SQL COMPLET - DZ-CarPool
-- Base de données PostgreSQL
-- =====

-- =====
-- 1. TABLES POUR LA GESTION DES UTILISATEURS
-- =====

-- Table des rôles
CREATE TABLE IF NOT EXISTS roles (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) UNIQUE NOT NULL,
    description TEXT DEFAULT ''
);

-- Table des préférences
CREATE TABLE IF NOT EXISTS preferences (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) UNIQUE NOT NULL,
    description TEXT DEFAULT '',
    name_fr VARCHAR(100) DEFAULT '',
    name_en VARCHAR(100) DEFAULT '',
    category VARCHAR(20) DEFAULT 'interests'
    CHECK (category IN ('interests', 'habits', 'driving')),
    icon VARCHAR(10) DEFAULT ''
);

CREATE INDEX idx_preferences_category
ON preferences(category, id);
```

DZ-CarPool - Script SQL

Script SQL - DZ-CarPool (Page 2/6)

```
-- Table des utilisateurs
CREATE TABLE IF NOT EXISTS users (
  id SERIAL PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  first_name VARCHAR(100) DEFAULT '',
  last_name VARCHAR(100) DEFAULT '',
  phone_number VARCHAR(20) DEFAULT '',
  phone_verified BOOLEAN DEFAULT FALSE,
  email_verified BOOLEAN DEFAULT FALSE,
  email_verified_at TIMESTAMP,
  phone_verified_at TIMESTAMP,
  profile_picture VARCHAR(255),
  bio TEXT,
  role_id INTEGER REFERENCES roles(id) ON DELETE SET NULL,
  is_active BOOLEAN DEFAULT TRUE,
  is_staff BOOLEAN DEFAULT FALSE,
  is_superuser BOOLEAN DEFAULT FALSE,
  date_joined TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  trips_as_driver INTEGER DEFAULT 0,
  trips_as_passenger INTEGER DEFAULT 0,
  average_rating NUMERIC(3, 2) DEFAULT 0.0,
  last_login TIMESTAMP
);

CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_date_joined ON users(date_joined);
```

DZ-CarPool - Script SQL

Script SQL - DZ-CarPool (Page 3/6)

```
-- Table de liaison users-preferences
CREATE TABLE IF NOT EXISTS users_preferences (
  id SERIAL PRIMARY KEY,
  user_id INTEGER NOT NULL
    REFERENCES users(id) ON DELETE CASCADE,
  preference_id INTEGER NOT NULL
    REFERENCES preferences(id) ON DELETE CASCADE,
  UNIQUE(user_id, preference_id)
);

CREATE INDEX idx_users_preferences_user
  ON users_preferences(user_id);
CREATE INDEX idx_users_preferences_preference
  ON users_preferences(preference_id);

-- Table des documents utilisateurs
CREATE TABLE IF NOT EXISTS user_documents (
  id SERIAL PRIMARY KEY,
  user_id INTEGER NOT NULL
    REFERENCES users(id) ON DELETE CASCADE,
  document_type VARCHAR(20) NOT NULL
    CHECK (document_type IN ('CNI', 'PERMIS', 'AUTRE')),
  file_path VARCHAR(255) NOT NULL,
  uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  verified BOOLEAN DEFAULT FALSE,
  verified_by_id INTEGER
    REFERENCES users(id) ON DELETE SET NULL,
  verified_at TIMESTAMP,
  rejection_reason TEXT DEFAULT ''
);
```

DZ-CarPool - Script SQL

Script SQL - DZ-CarPool (Page 4/6)

```
-- Table des refresh tokens
CREATE TABLE IF NOT EXISTS refresh_tokens (
  id SERIAL PRIMARY KEY,
  user_id INTEGER NOT NULL
    REFERENCES users(id) ON DELETE CASCADE,
  token_hash VARCHAR(255) UNIQUE NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  expires_at TIMESTAMP NOT NULL,
  revoked BOOLEAN DEFAULT FALSE
);

-- =====
-- 2. TABLES POUR LES TRAJETS
-- =====

-- Table des prix du carburant
CREATE TABLE IF NOT EXISTS fuel_prices (
  id SERIAL PRIMARY KEY,
  wilaya_code VARCHAR(2) NOT NULL,
  wilaya_name VARCHAR(100) NOT NULL,
  fuel_type VARCHAR(30) NOT NULL
    CHECK (fuel_type IN ('essence_sans_plomb',
                        'gasoil', 'gpl', 'electrique')),
  price_per_liter NUMERIC(10, 2) NOT NULL,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(wilaya_code, fuel_type)
);
```

DZ-CarPool - Script SQL

Script SQL - DZ-CarPool (Page 5/6)

```
-- Table des trajets
CREATE TABLE IF NOT EXISTS trajets (
  id SERIAL PRIMARY KEY,
  conducteur_id INTEGER NOT NULL
    REFERENCES users(id) ON DELETE CASCADE,
  ville_depart VARCHAR(100) NOT NULL,
  ville_arrivee VARCHAR(100) NOT NULL,
  adresse_depart TEXT NOT NULL,
  adresse_arrivee TEXT NOT NULL,
  date DATE NOT NULL,
  heure_depart TIME NOT NULL,
  nbr_places INTEGER NOT NULL CHECK (nbr_places > 0),
  places_disponibles INTEGER NOT NULL,
  price NUMERIC(10, 2) NOT NULL,
  price_platform NUMERIC(10, 2) NOT NULL,
  price_driver NUMERIC(10, 2) NOT NULL,
  distance NUMERIC(10, 2) NOT NULL,
  fuel_type VARCHAR(30) NOT NULL,
  fuel_consumption NUMERIC(5, 2) NOT NULL,
  wilaya_depart VARCHAR(100) NOT NULL,
  status VARCHAR(20) DEFAULT 'ACTIVE'
    CHECK (status IN ('ACTIVE', 'COMPLETED', 'CANCELLED')),
  description TEXT DEFAULT '',
  is_comfort BOOLEAN DEFAULT FALSE,
  CHECK (places_disponibles <= nbr_places)
);
```

DZ-CarPool - Script SQL

Script SQL - DZ-CarPool (Page 6/6)

```
-- Table des réservations
CREATE TABLE IF NOT EXISTS reservations (
  id SERIAL PRIMARY KEY,
  trajet_id INTEGER NOT NULL
    REFERENCES trajets(id) ON DELETE CASCADE,
  passenger_id INTEGER NOT NULL
    REFERENCES users(id) ON DELETE CASCADE,
  nbr_places INTEGER NOT NULL CHECK (nbr_places > 0),
  status VARCHAR(20) DEFAULT 'PENDING'
    CHECK (status IN ('PENDING', 'CONFIRMED',
                      'CANCELLED', 'REJECTED')),
  price_per_seat NUMERIC(10, 2) NOT NULL,
  total_price NUMERIC(10, 2) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  approved_at TIMESTAMP,
  cancelled_at TIMESTAMP,
  cancellation_reason TEXT
);

-- Trigger pour mettre à jour updated_at
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = CURRENT_TIMESTAMP;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

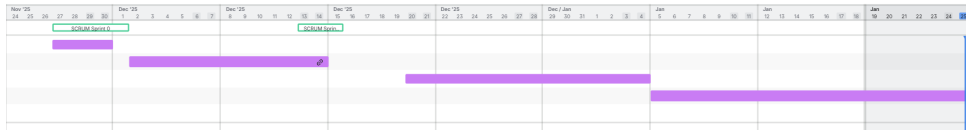


Fig. A.1 : Capture d'écran du backlog Jira - chronologie du sprint

Vue montrant la chronologie des activités et états du sprint.

✓ Tickets enfant ... ▢ +

Progression : 100 %

Tickets	Pri...	Per...	État
SCRUM-6 -Diagramme cas d'utilisation / Diagrammes de séquence...	^ H..	OL O	TERMINÉ(E) ▾
SCRUM-12 -Diagrammes de séquence (authentification complète)	^ H..	YD Y	TERMINÉ(E) ▾
SCRUM-13 Diagrammes de séquence (search + search avec filter)	^ H..	A a	TERMINÉ(E) ▾
SCRUM-14 Design (5 pages minimum)	^ H..	M rr	TERMINÉ(E) ▾
SCRUM-15 Organisation du Travail sous Jira	^ H..	RB R	TERMINÉ(E) ▾
SCRUM-21 rapport de sprint	= M.	RB R	TERMINÉ(E) ▾
SCRUM-23 Diagramme cas d'utilisation	= M.	OL O	TERMINÉ(E) ▾
SCRUM-24 Schéma de la Base de données	= M.	RB R	TERMINÉ(E) ▾
SCRUM-25 diagramme de classe	= M.	OL O	TERMINÉ(E) ▾
SCRUM-26 Rapport de sprint 0	^ H..	RB R	TERMINÉ(E) ▾

Fig. A.2 : Capture d'écran d'un backlog Jira - Détail du sprint 1

Détail des user stories et tâches associées au sprint 1.

✓ Tickets enfant ... ▢ +

Progression : 100 %

Tickets	Pri...	Per...	État
SCRUM-17 Vérification de l'avancement du Frontend	= M.	OL O	TERMINÉ(E) ▾
SCRUM-22 Rapport de sprint	= M.	OL O	TERMINÉ(E) ▾
SCRUM-18 Vérification de l'avancement du Backend	= M.	OL O	TERMINÉ(E) ▾
SCRUM-27 Diagramme de Composants UML	= M.	YD Y	TERMINÉ(E) ▾
SCRUM-28 ARCHITECTURE_DECISIO NS.md	= M.	RB R	TERMINÉ(E) ▾

Fig. A.3 : Capture d'écran d'un backlog Jira - Détail du sprint 2

Détail des user stories et tâches associées au sprint 2.

▼ Tickets enfant ... ▢ +

Progression : 100 %

Tickets	Pri...	Per...	État
SCRUM-32 ajouter la fonction de vérification d'email et...	⤴ H..	RB R	TERMINÉ(E) ▼
SCRUM-33 frontend (profile,admin,notification...	⤵ L..	RB R	TERMINÉ(E) ▼
SCRUM-34 linker le backend et le frontend et perfectionner...	= M.	RB R	TERMINÉ(E) ▼
SCRUM-35 rapport finale GL	= M.	OL O	TERMINÉ(E) ▼
SCRUM-36 rapport BDD	= M.	YD Y	TERMINÉ(E) ▼
SCRUM-37 tests et intégration	⤵ L..	RB R	TERMINÉ(E) ▼
SCRUM-38 terminer design	⤴ H..	M IT	TERMINÉ(E) ▼
SCRUM-40 google auth	⤴ H..	RB R	TERMINÉ(E) ▼
SCRUM-42 page preferences front et back linker	⤴ H..	RB R	TERMINÉ(E) ▼
SCRUM-43 rapport-du-sprint	= M.	A a	TERMINÉ(E) ▼

Fig. A.4 : Capture d'écran d'un backlog Jira - Détail du sprint 3

Détail des user stories et tâches associées au sprint 3.

▼ SCRUM Sprint 4 (Ajouter des dates: 3 tickets) 5 6 7 Démarrer un sprint ...

SCRUM-49 faire une video youtube	PRESENTATION	TERMINÉ(E) ▼	2	⤵
SCRUM-50 deploiement de la plateforme	PRESENTATION	TERMINÉ(E) ▼	4	⤵
SCRUM-51 finalisation de rapport BD	PRESENTATION	ENCOURS	2	⤵

+ Créer

Fig. A.5 : Capture d'écran d'un backlog Jira - Détail d'un sprint 4

Détail des user stories et tâches associées au sprint 4.

Lien du dépôt GitHub :

[https ://github.com/Rayanebsh/DZ-CarPool](https://github.com/Rayanebsh/DZ-CarPool)